WISEConv: Worklist-Driven Masked Convolution for Low-Latency Vision Inference

Anonymous Author(s)

Abstract

Masked convolution promises to reduce vision-model inference latency by computing only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to translate sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime built on the observation that output coordinates form the interface between position selection and the regular reduction datapath in the dense tiled pipeline. WISEConv replaces the dense tiled pipeline's contiguous coordinate source with an active-output worklist, restricting computation to almost exclusively active positions while retaining dense tensor layouts and the regular reduction datapath. To keep worklist construction cheap and sustain throughput as the number of active outputs changes per frame, WISEConv fuses mask propagation with tile-local compaction, reuses coordinate metadata across compatible operators, and adapts its compute decomposition without host synchronization, securing both a high useful-compute ratio and high throughput. We evaluate WISEConv on four perception models across six GPU platforms: four discrete GPUs and two Jetson platforms. WISEConv achieves a 1.57× geometric-mean speedup over the fastest competing path per model-platform pair, reducing full-model latency by 22.1–46.5%. On the Jetson platforms, it also reduces per-frame board energy by 28.5–72.6% relative to dense execution.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates per-frame inference latency in many latency-critical vision tasks [3, 10, 40], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks spatially active positions on the dense feature grid, and a dedicated sparse convolution operator updates only these positions in the dense output. The mask may derive from event-camera increments [8, 24], temporal differences [17, 30], or learned spatial gates [22, 37]. These mask sources promise large reductions in convolution work, yet existing masked-convolution systems [8, 22, 30, 37] rarely translate the sparsity into proportional latency reduction.

Dense GPU convolution obtains high *throughput* (operations per second) from coordinate-driven, tiled execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath with predictable memory access. In masked convolution, reducing latency proportionally requires sustaining this throughput while achieving a high *useful-compute ratio* (the fraction of issued operations that produce needed outputs), but the per-frame position selection imposed by the mask conflicts with the contiguous coordinate assumption that sustains this throughput, placing the two requirements in tension.

Existing systems either retain the tiled execution for high throughput or eliminate wasted work for a high useful-compute ratio, but not both. *Tile skipping* [8, 17, 30, 32] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [22, 37, 41] instead extracts exactly the active positions into a packed representation, computes, and scatters the results back, incurring extraction, irregular access, and writeback overhead that damages throughput.

Neither path converts sparsity into proportional latency reduction (§2.2). In our evaluation, under a temporal difference mask policy [30], 52% of YOLOv8n [19]'s convolution work is unnecessary, yet tile skipping [30] incurs 26.1% higher latency than dense with a useful-compute ratio of only ??%; gather-scatter [5] lifts the useful-compute ratio to near one but at 4.3× the latency of dense. Even on the sparser FireFlowNet [28] under event-camera increments [8], where 75.1% of work is unnecessary and tile skipping does cut latency by 42.0%, it still wastes 26.4% of its issued operations; gather-scatter sustains only ??% of tile skipping's throughput, at 2.0× the latency of dense.

Our key insight is that output coordinates form the interface between position selection and the reduction datapath in tiled execution. Replacing the contiguous coordinate source with an active-output worklist changes only which positions are scheduled, while preserving the dense tensor layouts and regular reduction datapath that sustain throughput. The worklist interface makes it possible to compute almost exclusively required outputs without abandoning the dense pipeline's throughput potential.

Realizing this potential requires solving two challenges. First, *worklist construction cost*: construction scans the mask, so its cost scales with the layer's spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction's share of latency therefore grows as activity falls and channels thin, and accumulates

across operators when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist's order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full, and because the length is device-resident, any host-side decision that depends on it costs a synchronization on the per-frame path. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-dr**I**ven ma**S**k**E**d **Conv**olution), a masked-convolution runtime that fuses position-level selectivity into the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. To reduce worklist construction cost, a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse valid metadata instead of reconstructing it.

To sustain end-to-end worklist efficiency, construction emits contiguous per-tile segments ordered by the dense tile traversal, giving the compute stage tile-local structure without global sorting. Because the worklist changes the number of output-coordinate rows while each row's tensor layout and reduction datapath remain unchanged, WISEConv launches a persistent grid that serves any worklist length, with idle thread blocks absorbing reduction work from under-subscribed tiles. The kernel reads the device-resident worklist length and indexes a precomputed table and decomposes the workload accordingly, sustaining GPU utilization across varying worklist lengths without host synchronization.

This paper makes the following contributions:

- We characterize existing masked-convolution systems along two axes, useful-compute ratio and throughput, exposing why these systems do not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into the dense tiled convolution pipeline and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across six GPU platforms: four discrete GPUs and two Jetson platforms. WISEConv is the fastest path in all 24 workload-platform pairs, achieving a 1.57× geometric-mean speedup over the fastest evaluated competing path in each pair (22.1–46.5% lower full-model latency) and reducing per-frame board energy by 28.5–72.6% relative to dense execution on the Jetson platforms.

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [3, 34, 38, 40, 44], where per-frame inference latency bounds system responsiveness [9–11, 18]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [8, 24], temporal residuals between video frames [4, 17, 29, 30, 39], or learned input-dependent gates [13, 22, 37, 41]. The event and temporal policies mark positions that changed since the previous frame; the learned gates mark positions the task treats as relevant. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution keeps the tensors and arithmetic of dense convolution and restricts only which output coordinates it computes. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{f} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

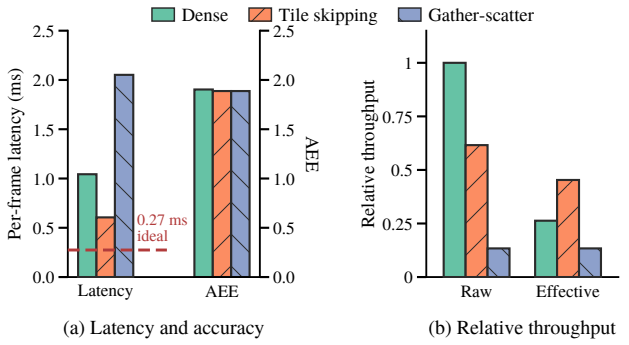
$$M_{out}[p] = \begin{cases} 1, & \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{f}$.

An execution path issues computation over a sequence of position slots C, where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non-$\perp$ slot $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{x}[\mathcal{N}_i(p)]\, \mathbf{f}_i, \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{f}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. How unrequested positions are represented or merged into a dense result is policy-specific. Event and temporal execution retains prior state, whereas learned gating merges the computed branch values into a residual or bypass tensor. Coverage is therefore the operator-level correctness condition: every position requested by the policy is computed, while the policy determines the treatment of all other positions. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and required work, whereas the execution path determines the issued sequence C.



(a) Latency and accuracy (b) Relative throughput

Figure 1. FireFlowNet [28] using the Ev-Conv [8] event-driven mask policy on MVSEC [45] and RTX 3080, comparing cuDNN (Dense), DeltaCNN tile skipping [30], and MMCV's native gather-scatter path [5]. (a) Mean full-model per-frame latency and average endpoint error (AEE; lower is better). The dashed line marks dense latency scaled by the active ratio aggregated across layers; the sparse paths use the same masks and produce nearly identical AEE. (b) Raw throughput T and effective throughput T_{eff}, each normalized to dense raw throughput. The dense useful-compute ratio equals the active ratio set by the Ev-Conv mask policy ($\alpha \approx 26.3\%$).

2.2 The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [8, 17, 23, 29, 32, 39] inherits the dense pipeline's tile-based computation without additional metadata passes, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [30] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [22, 41] instead abandons the dense pipeline: it extracts exactly the active coordinates into a packed representation, runs a separate GEMM, and scatters the results back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [37] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

A policy fixes the active ratio α, defined as the fraction of output-grid positions marked active, whereas a concrete execution path determines the useful-compute ratio η, defined as the fraction of issued slots that correspond to required outputs:

$$\alpha = \frac{\|M_{out}\|_0}{|\mathcal{G}|}, \quad \eta = \frac{\|M_{out}\|_0}{|C|}. \tag{3}$$

For $|C| > 0$, η quantifies schedule efficiency by capturing the fraction of issued computation that is useful; the remaining slots correspond to inactive or padding positions. An ideal execution path achieves $\eta = 1$ by issuing only required outputs; dense convolution, which issues the entire grid, has $\eta \approx \alpha$. An execution path also determines its throughput T, which combines with η to yield the effective throughput T_{eff}:

$$T = \frac{2|C|k^2 C_{in} C_{out}}{t}, \quad T_{\text{eff}} = \eta T. \tag{4}$$

where t denotes the latency of the execution path, including auxiliary mask and coordinate processing, k is the convolution kernel size, and C_{in} and C_{out} are the input and output channel counts of the masked convolution layer, respectively. The factor of two counts each multiply-accumulate as two FLOPs. Consequently, T_{eff} directly characterizes the processing rate of required outputs and therefore determines the realized latency.

Figure 1 traces this on RTX 3080 for FireFlowNet [28] under the Ev-Conv [8] mask policy. Event cameras expose microsecond-scale changes, and Ev-Conv evaluates optical flow on event-window inputs updated at 1 kHz. The policy marks 73.7% of the convolution work unnecessary, and the sparse paths discard it without reducing accuracy. Scaling dense latency by the fraction of convolution work that remains gives a 0.274 ms ideal proportional-scaling reference (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [30]) reaches 0.606 ms, still 2.21× the reference, and gather-scatter (MMCV [5]) runs at 2.052 ms, almost twice the latency of dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput at 61.6% of dense but at a useful-compute ratio of only 73.6%; gather-scatter lifts the useful-compute ratio to one, yet extraction and data movement collapse its throughput to 13.4% of dense (Fig. 1b). Neither path efficiently converts the GPU's capability into effective throughput.

2.3 Distinction from Related Sparse Execution

Other related sparse-execution work differs from WISEConv in representation, operator, sparsity type, or platform. 3D sparse-convolution systems [7, 16, 33, 35, 36, 42] operate on sparse point-cloud coordinates and construct coordinate maps for irregular input-to-output relations; WISEConv instead consumes a runtime mask over a dense 2D grid while retaining dense tensor layouts. Sparse matrix multiplication [12, 14, 31] accelerates sparse-graph and pruned-weight matrix products, where sparsity is structural rather than a dynamic spatial mask over a convolution grid. Structured weight pruning (e.g., 2:4 sparsity [26]) exploits fixed patterns in model weights. Sparse-format compilers [21, 43] expose sparsity through explicit tensor formats, whereas WISEConv retains dense layouts and uses a transient per-frame mask only to select output coordinates. DPACS [15] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

Some 3D systems [33, 36] compute through coordinate-indexed fused GEMM [1], where an index array derived from coordinate maps redirects the position dimension of a tiled GEMM. It seemingly coincides with WISEConv's goal, but 3D sparse convolution requires explicit coordinate metadata

because its tensors are represented as sparse coordinate sets. Constructing the index array incurs hashing, sorting, and global scans that frequently dominate per-layer latency over the GEMM itself [36, 42], making construction cost one of the central research problems in the 3D community; thus directly introducing coordinate indirection into 2D convolution which already tiles densely appears unattractive. 2D masked convolution, however, presents a structurally simpler mapping problem: the output grid is regular, the mask is a bitmap, and each coordinate's receptive-field addresses follow from fixed grid arithmetic, so the coordinate-map machinery of 3D systems is unnecessary. For 2D masked convolution, WISEConv identifies the coordinate source as the replaceable interface between position selection and tiled reduction, while the co-design of worklist construction and consumption realizes this insight to secure both a high useful-compute ratio and high throughput.

3 WISEConv Overview

Dense tiled convolution already consumes a stream of output coordinates: each coordinate determines the receptive-field reads and output write, while its reduction over channels and kernel offsets remains regular. WISEConv exploits this interface by replacing the dense coordinate sequence with a worklist of requested output positions, thereby introducing position-level selectivity into the tiled pipeline. Removing unrequested positions from the schedule raises the useful-compute ratio, but issuing fewer operations alone does not guarantee lower latency. To translate selectivity into latency reduction, the worklist must be inexpensive to construct and must be consumed with both a high useful-compute ratio and high throughput. WISEConv therefore co-designs a construction stage and a compute stage around these interdependent requirements, as Figure 2 summarizes.

Construction stage. To prevent worklist construction from erasing the savings of selective compute, WISEConv confines construction to mask and coordinate metadata (Figure 2d, ①), never reading feature or weight tensors. In one mask-space pass, it fuses the output-mask propagation in Eq. 1 with tile-local compaction, producing M_{out} and a worklist $\mathcal{W}$ (the figure shows the compaction step; mask propagation precedes it but is omitted); the requested coordinates from each spatial tile form a segment in local dense-traversal order, preserving spatial grouping without global ordering. WISEConv stores $\mathcal{W}$ in a buffer preallocated for $|\mathcal{G}| = H_{out}W_{out}$ coordinates, the static upper bound on $|\mathcal{W}|$. A device-resident counter records $n_{\mathcal{W}} = |\mathcal{W}|$, so construction needs neither runtime allocation nor a host-visible active count.

Because construction cost scales with the spatial grid rather than the active count or channel widths, its share of latency grows as activity falls and accumulates when every layer rebuilds. WISEConv therefore propagates and reuses the resulting mask and worklist metadata across compatible operators instead of reconstructing it, amortizing one construction across the compatible operator sequence. The fused pass lowers the cost of each construction, while cross-operator reuse reduces how often it occurs; together, they address the Worklist Construction Cost challenge.

Compute stage. The compute stage (Figure 2d, ②) replaces only the coordinate source of a dense tiled implicit-GEMM pipeline with the worklist; tensor layouts, weight access, and the per-position channel-and-kernel reduction remain unchanged from the dense path. The worklist length varies across layers and inputs and remains device-resident. A fixed decomposition cannot serve the full range: too little parallelism leaves the GPU underutilized at low activity, while unnecessary parallelism adds overhead at high activity, both degrading throughput. Yet the conventional approach of selecting a workload-size-specific decomposition on the host is impractical, because reading the length back would incur a synchronization on the critical path.

The worklist length varies per frame with the input mask: the tile count is unknown before launch and device-resident after construction. Synchronizing it to the host for decomposition selection would insert a per-operator stall on the critical path, yet a fixed decomposition loses efficiency across the resulting range of tile counts. WISEConv builds on Stream-K [27], which addresses GPU underutilization when tiles cannot fill all SMs but assumes the tile count is host-visible at launch. WISEConv designs *on-device adaptive Stream-K execution*: it launches a persistent grid sized to SM residency, and the kernel reads the device-resident worklist length, looks up an offline-calibrated policy table, and selects its decomposition per frame without host involvement. This sustains GPU throughput as the worklist length varies from frame to frame.

Neither stage suffices alone: excessive construction cost, a low useful-compute ratio, or low consumption throughput can each erase the benefit of issuing less convolution work. By controlling all three jointly, WISEConv converts upstream spatial sparsity into end-to-end latency reduction.

4 WISEConv Design

The design follows the worklist from its construction (§4.1) and propagation across compatible operators (§4.2) to its consumption by the convolution pipeline (§4.3).

4.1 Tile-Local Worklist Construction

Constructing a tight schedule already requires a scan of the output grid, but the resulting coordinates must also retain enough spatial structure for efficient consumption. Globally stable compaction preserves the complete dense traversal order but requires a prefix scan or another pass to obtain device-wide offsets; per-coordinate appends avoid that coordination but scatter neighboring positions. Materializing

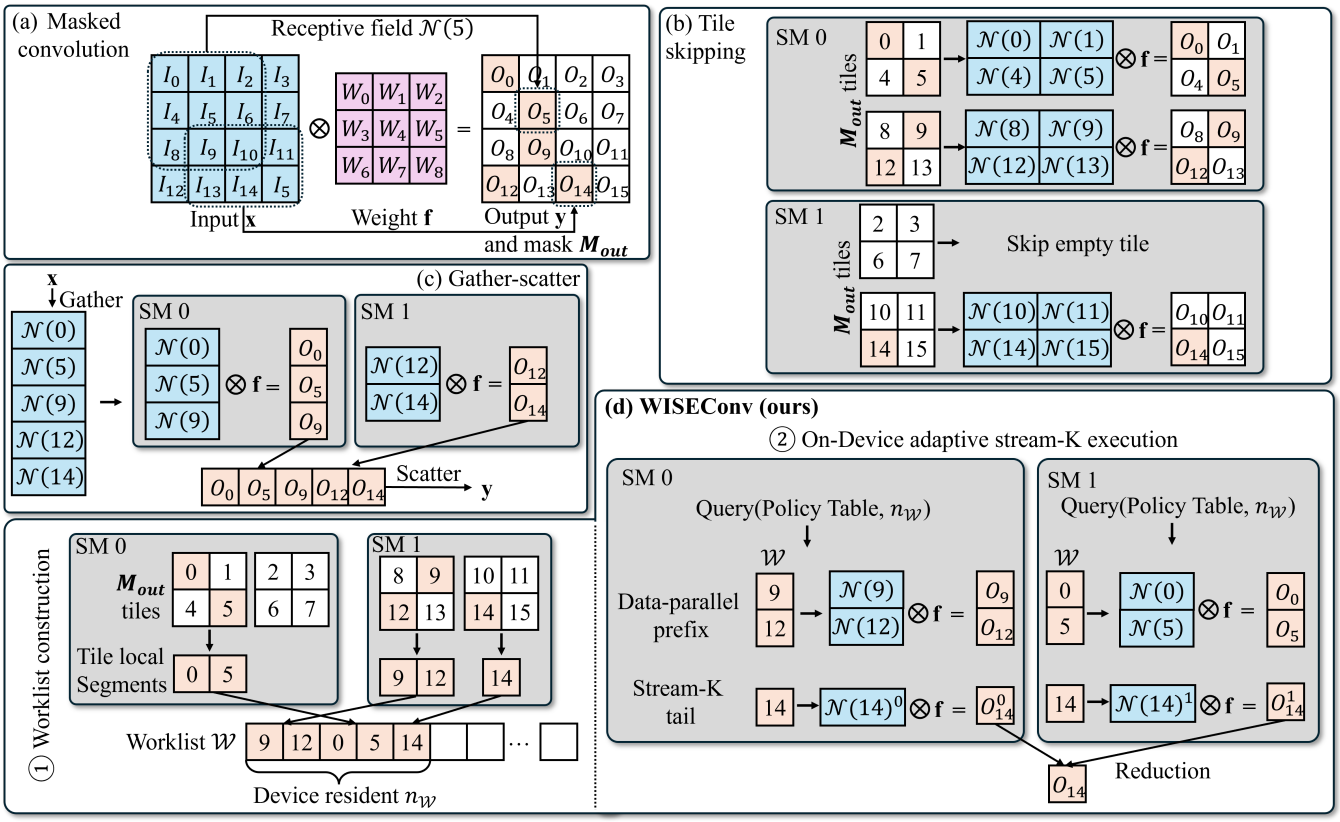


Figure 2. Overview of WISEConv and baseline execution paths. (a) Masked convolution: M_{out} marks active output positions (colored); the operator computes only these from **x** and **f**. (b) Tile skipping skips only fully inactive tiles; partially active tiles compute all positions. (c) Gather-scatter packs exactly the receptive fields of active positions, computes, and scatters results back. (d) WISEConv: ① constructs a tile-local worklist and ② consumes it via on-device adaptive Stream-K execution.

active receptive fields, as gather-scatter does, regularizes the compute input at the cost of feature traffic proportional to the active count and $k^2 C_{in}$. The useful middle ground is therefore a metadata-only schedule with local rather than global ordering.

WISEConv realizes this middle ground with contiguous, tile-local worklist segments. Because the number of active positions is not known before the pass, it preallocates a coordinate buffer with capacity $|\mathcal{G}| = H_{out} W_{out}$, the static upper bound on $|\mathcal{W}|$. Algorithm 1 details the resulting single pass. Line 1 initializes the device-resident length counter. Lines 2–7 partition the output grid into $B_\tau \times B_\tau$ construction tiles, derive the corresponding entries of M_{out} using Eq. 1, and compact each tile's active coordinates into a local sequence $\mathcal{P}_\tau$ in dense-traversal order. When a learned gate supplies M_{out} directly [22, 37], Lines 4–5 are omitted and Line 7 compacts the supplied mask. Lines 8–11 count the compacted coordinates, atomically reserve $[\text{base}, \text{base} + n_\tau)$ from the counter, and copy $\mathcal{P}_\tau$ into that interval. After all tiles complete, the reserved intervals collectively store $\mathcal{W}$, and the counter records $n_{\mathcal{W}}$. Because the layer shape fixes the buffer capacity and launch geometry while only the device observes the data-dependent length, construction requires no runtime allocation or host-visible active count.

To relate the worklist to the useful-compute ratio of Eq. 3, we insert its length $n_{\mathcal{W}}$ between the required-position count and the issued-slot count. For $n_{\mathcal{W}} > 0$,

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{\text{construct}}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{|C|}}_{\eta_{\text{compute}}} . \tag{5}$$

Here, $\eta_{\text{construct}}$ compares required positions with worklist entries, while η_{compute} compares worklist entries with issued slots. We analyze the former here and the latter after defining the compute organization in §4.3. The factorization decomposes η only; all stage costs remain reflected in throughput T.

Because construction tiles partition the output grid and use disjoint reservations, each active coordinate appears exactly once, so $n_{\mathcal{W}} = \|M_{out}\|_0$. Fresh construction therefore has $\eta_{\text{construct}} = 1$ for a nonempty worklist. Coverage constrains membership rather than order, so tile reservation

Algorithm 1 Tile-Local Worklist Construction

Require: Input mask M_{in}, output grid $\mathcal{G}$, tile size B_τ
Ensure: Mask M_{out}, worklist $\mathcal{W}$, length $n_{\mathcal{W}} = |\mathcal{W}|$

$n_{\mathcal{W}} \leftarrow 0$
for each $B_\tau \times B_\tau$ spatial tile τ **in parallel do**
  for each position $p \in \tau$ **in parallel do**
    active$(p) \leftarrow \exists\, r \in \mathcal{N}(p) : M_{in}[r] = 1$
    $M_{out}[p] \leftarrow$ active(p)
  end for
  $\mathcal{P}_\tau \leftarrow$ Compact(τ, M_{out})
  $n_\tau \leftarrow |\mathcal{P}_\tau|$
  base $\leftarrow$ AtomicAdd$(n_{\mathcal{W}}, n_\tau)$
  for each $0 \le j < n_\tau$ **in parallel do**
    $\mathcal{W}[\text{base} + j] \leftarrow \mathcal{P}_\tau[j]$
  end for
end for

order does not affect correctness; traversal order within each interval is retained for locality.

WISEConv thus confines its ordering guarantee to each contiguous segment: coordinates retain the tile's dense traversal, while independently reserved segments provide no cross-boundary locality guarantee. A compute tile of B_M positions (the output-position tile width used by the compute stage, §4.3) may therefore span coordinates from multiple construction segments when segments are shorter than B_M; the preserved locality is intra-segment contiguity, not compute-tile-to-construction-tile alignment. As shown in §6.4, this tile-local order achieves compute-stage throughput comparable to global ordering, making the additional ordering pass an unfavorable end-to-end trade-off.

4.2 Cross-Operator Worklist Reuse

Each construction pass examines all $H_{out}W_{out}$ positions and, when deriving M_{out} from M_{in}, performs $O(k^2)$ mask work per position. This channel-independent grid scan contrasts with required convolution work, which scales with the active ratio, spatial extent, and both channel dimensions. Repeating the scan across same-grid layers that introduce no new required positions is therefore redundant. WISEConv instead amortizes construction over each compatible layer sequence by reusing one worklist across its operators.

For layer l, let $\mathcal{G}^l$ denote its output grid, M^l_{out} its output mask, $\mathcal{W}^l$ its worklist, and $n^l_{\mathcal{W}}$ its device-resident length. WISEConv propagates the mask and worklist metadata alongside the feature tensor. Whether two consecutive layers are reuse-compatible depends on their operator types, not on runtime mask values. An operator that preserves or narrows the output grid and active set guarantees, for any input mask, that $\mathcal{G}^{l+1} = \mathcal{G}^l$ and

$$\{p \in \mathcal{G}^{l+1} \mid M^{l+1}_{out}[p] = 1\} \subseteq \{p \in \mathcal{G}^l \mid M^l_{out}[p] = 1\}. \quad (6)$$

WISEConv checks this condition once at model-conversion time. Because $\mathcal{W}^l$ covers M^l_{out}, Eq. 6 guarantees coverage for M^{l+1}_{out}. WISEConv therefore forwards $\mathcal{W}^{l+1} = \mathcal{W}^l$ and $n^{l+1}_{\mathcal{W}} = n^l_{\mathcal{W}}$ without reconstruction and applies the condition recursively at each subsequent layer.

Equality in Eq. 6 covers same-grid 1×1 convolutions, mask-preserving activations, and inference-time normalization, leaving $\eta_{\text{construct}}$ unchanged. Strict inclusion can result from mask-narrowing operators in the upstream policy, such as the activation-fused update truncation that DeltaCNN uses to increase downstream sparsity [30]. The inherited worklist remains a valid superset, trading a lower $\eta_{\text{construct}}$ for one fewer construction pass without violating coverage. Because compute gates all output writes by the exact mask, positions scheduled but no longer active under the current mask participate in reduction yet produce no store; the superset schedule therefore cannot alter any inactive position's observable state.

A change in the output grid violates the geometry constraint, while receptive-field expansion may violate Eq. 6 by introducing positions outside the upstream mask; either case invokes the construction pass of §4.1 to emit a new tight worklist. A lower $\eta_{\text{construct}}$ alone does not trigger reconstruction because tightening a valid superset would pay another grid scan. Common receptive-field-expanding layers such as 3×3 convolutions still create rebuild points throughout the network, leaving device-resident worklist lengths variable across layers and frames. Compute must therefore handle a dynamic problem size.

4.3 On-Device Adaptive Stream-K Execution

Construction and reuse fix $\eta_{\text{construct}}$; the compute stage controls η_{compute} and throughput T. Dense convolution maps to an implicit GEMM with M output positions, $N = C_{out}$ output channels, and reduction depth $K = k^2 C_{in}$. An output tile covers B_M positions and B_N output channels; the reduction over K proceeds in steps of B_K. In the standard *data-parallel* decomposition, each of the $\lceil M/B_M \rceil \times \lceil N/B_N \rceil$ output tiles is assigned to one thread block, which performs the full K reduction; split-K instead replicates each tile across S_k blocks that each reduce a disjoint K-slice. Because M, N, and K are static, the tile shape (B_M, B_N), reduction step B_K, and split factor can be selected once offline by profiling and keeping the fastest.

In worklist convolution, N and K remain static, but the M dimension becomes $n_{\mathcal{W}}$, the device-resident worklist length written by construction, which varies by an order of magnitude or more across frames of a single layer. The offline assumption breaks: a single configuration cannot serve the full range. Two failure regimes bracket the problem. When activity is high and output tiles are plentiful, K-axis parallelism adds coordination and accumulation overhead without improving occupancy. When activity is low and tiles

are scarce, data-parallel execution leaves streaming multiprocessors (SMs) without work because the tile count no longer fills a full *wave* (the set of thread blocks that co-reside across all SMs), referred to as the problem of wave quantization. Adapting between these regimes requires knowing $n_{\mathcal{W}}$; transferring it to the host inserts a per-operator synchronization on the critical path.

WISEConv builds on Stream-K [27], the established solution to wave quantization in dense GEMM. Stream-K launches a *persistent grid* of g_{res} blocks determined by the device's SM occupancy (not the tile count) and partitions all tiles' K-iterations into S_k units consumed by grid stride; partial K-slices fill SMs that would otherwise sit idle after the data-parallel waves complete. Its hybrid variant [1] restricts the K-splitting to the partial tail wave, handling complete waves data-parallel to avoid partial-reduction overhead where it is unnecessary. In worklist convolution the tile count changes every frame, so the wave boundary shifts with it. This shifting boundary is the operating point of WISEConv's *on-device adaptive Stream-K execution*: the compute kernel reads the exact worklist length on the device, determines the current wave boundary, and selects between data-parallel, Stream-K, and hybrid Stream-K decomposition accordingly, without host involvement.

Because the host cannot observe $n^l_{\mathcal{W}}$ without synchronization, WISEConv fixes the tile configuration $\hat{q}^l = (B_M, B_N, B_K)$ per layer at deployment and adapts only the decomposition mode on-device. What changes per frame is how the K reduction across output tiles is assigned to thread blocks, selected from three modes:

- *Data-parallel*: each block owns a complete output tile and reduces the full K range.
- *Stream-K*: each output tile's K-iterations are partitioned into S_k units; the persistent grid consumes all units by grid stride, and partial results from blocks sharing a tile are resolved through the workspace.
- *Hybrid Stream-K*: a data-parallel prefix handles complete waves; a streaming tail distributes the remaining tiles' K-iterations across one wave of blocks, eliminating partial-wave waste without applying the streaming overhead to the entire grid.

Both $\hat{q}^l$ and the per-length mode choice are calibrated offline. WISEConv selects $\hat{q}^l$ from a small catalog $\mathcal{Q}$ by profiling each configuration at the full worklist $\mathcal{W}^l_{\text{full}}$ (all $|\mathcal{G}^l|$ output positions) and keeping the fastest. Let $t(q, \mathcal{W})$ denote the minimum kernel latency of configuration q on worklist $\mathcal{W}$ over all mode candidates:

$$\hat{q}^l = \arg\min_{q \in \mathcal{Q}} \; t(q,\ \mathcal{W}^l_{\text{full}}). \tag{7}$$

At the full worklist $\eta_{\text{compute}} \approx 1$, so this step essentially maximizes throughput T, ensuring competitiveness with dense execution. The choice of B_M also determines η_{compute} at every other worklist length: the issued slot count is $|C| = B_M\lceil n^l_{\mathcal{W}}/B_M\rceil$, so $\eta_{\text{compute}} = n^l_{\mathcal{W}}/|C|$ can drop as $n^l_{\mathcal{W}}$ shrinks relative to B_M. Fixing B_M per layer is a necessary constraint, but per-shape profiling partially mitigates the cost: a large B_M on a small output grid often produces too few tiles to fill the device even at full worklist, so such layers naturally select a smaller B_M that also limits η_{compute} loss at low activity. The mode table below compensates for the remaining throughput gap by adapting the K-splitting strategy on-device.

Within $\hat{q}^l$, WISEConv builds a *policy table* P^l indexed by tile count $s = \lceil n^l_{\mathcal{W}}/B_M\rceil \times \lceil C_{out}/B_N\rceil$. For each s it profiles every candidate mode m and split factor S_k:

$$P^l[s] = \arg\min_{m,\, S_k} \; t(\hat{q}^l,\ s,\ m,\ S_k), \tag{8}$$

where the minimization ranges over the three modes and their compatible split factors. All three modes process the same output tiles and differ only in how K-iterations are assigned to blocks, so η_{compute} is invariant across modes and minimizing latency directly maximizes throughput T, and hence $\eta_{\text{compute}}\, T$.

Algorithm 2 presents the hybrid mode, which subsumes the other two: data-parallel execution corresponds to $s_{\text{dp}} = s$ (empty tail), while pure Stream-K corresponds to $s_{\text{dp}} = 0$ (no data-parallel prefix); the policy table selects among the three based on the current tile count. All three modes share the same gather-at-load reduction datapath; only the block-to-work mapping differs. Algorithm 2 omits boundary-tile padding when $n_{\mathcal{W}}$, C_{in}, or C_{out} is not divisible by B_M, B_K, or B_N, respectively; the implementation masks out-of-bounds accesses in these tiles.

At runtime, the kernel reads $n_{\mathcal{W}}$, derives the tile count, and looks up the decomposition mode from the policy table P^l (Lines 1–4 in Algorithm 2). The persistent grid first processes complete waves data-parallel (Lines 7–18): each block decodes its position and channel tile indices, reads B_M output coordinates from the worklist $\mathcal{W}$, and reduces the full reduction depth, writing the result directly to the output. The remaining partial-wave tiles enter the Stream-K tail (Lines 21–36): the reduction depth of each tile is partitioned among S_k contributors (the split factor selected by the policy table), and each block reduces its assigned slice into a pre-allocated workspace slot for that tile. Each slot holds a $B_M \times B_N$ partial accumulator in accumulation precision (typically FP32); the workspace $\mathbf{Z}$ is sized to $\max(S_k) \times |\mathbf{y}^l|$ elements per layer, where $\max(S_k)$ is the largest split factor in the policy table P^l. A per-tile completion counter tracks how many contributors have finished; the last block to arrive loads all partial fragments, reduces them, applies the output epilogue, and writes the final result (Lines 33–37). Both write paths guard stores with the exact output mask M_{out}, so positions present in a reused superset worklist but no longer active under the current mask produce no output store. No device-wide barrier is needed; data-parallel tiles bypass the workspace entirely and write output directly.

Algorithm 2 On-Device Adaptive Hybrid Stream-K

Require: $\mathcal{W}$, M_{out}, P, g_{res}, B_M, B_N, B_K, $\mathbf{x}$, $\mathbf{f}$, k, $\mathbf{Z}$
Ensure: $\mathbf{y}$

$n_{\mathcal{W}} \leftarrow$ **read** device-resident worklist length
$s_M \leftarrow \lceil n_{\mathcal{W}}/B_M \rceil$; $s_N \leftarrow \lceil C_{out}/B_N \rceil$; $s \leftarrow s_M \times s_N$
$S_k \leftarrow P[s]$ ▷ *policy lookup*
$K_{\text{iters}} \leftarrow k^2\lceil C_{in}/B_K \rceil$
▷ ——————————*Data-parallel prefix*——————————
$s_{\text{dp}} \leftarrow s - (s \bmod g_{\text{res}})$ ▷ *full-wave boundary*
for $b = \text{bid},\ \text{bid}+g_{\text{res}},\ \ldots < s_{\text{dp}}$ **do**
  $(b_m, b_n) \leftarrow \textsc{Decode}(b,\ s_N)$
  $\mathbf{w} \leftarrow \mathcal{W}[b_m B_M\ ..\ (b_m{+}1)B_M)$ ▷ *worklist indirection*
  $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
  for $j = 0, \ldots, K_{\text{iters}}{-}1$ **do**
    $(i, c_k) \leftarrow \textsc{Offset}(j)$ ▷ *kernel offset, channel tile*
    $\mathbf{X} \leftarrow \mathbf{x}[\mathcal{N}_i(\mathbf{w}),\ c_k B_K{:}(c_k{+}1)B_K)$
    $\mathbf{F} \leftarrow \mathbf{f}_i[c_k B_K{:}(c_k{+}1)B_K,\ b_n B_N{:}(b_n{+}1)B_N)$
    $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{F}$
  end for
  $\mathbf{y}[\mathbf{w},\ b_n B_N{:}(b_n{+}1)B_N) \leftarrow \mathbf{Y}$ **where** $M_{out}[\mathbf{w}]{=}1$
end for
▷ ——————————*Stream-K tail*——————————
$s_{\text{tail}} \leftarrow s - s_{\text{dp}}$
for $b = \text{bid},\ \text{bid}+g_{\text{res}},\ \ldots < s_{\text{tail}} \times S_k$ **do**
  $(b_m, b_n, r) \leftarrow \textsc{TailDecode}(b,\ s_N,\ S_k,\ s_{\text{dp}})$
  $(k_{\text{lo}}, k_{\text{hi}}) \leftarrow \textsc{Partition}(K_{\text{iters}},\ S_k,\ r)$ ▷ *r-th K-slice*
  $\mathbf{w} \leftarrow \mathcal{W}[b_m B_M\ ..\ (b_m{+}1)B_M)$
  $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
  for $j = k_{\text{lo}}, \ldots, k_{\text{hi}}{-}1$ **do**
    $(i, c_k) \leftarrow \textsc{Offset}(j)$
    $\mathbf{X} \leftarrow \mathbf{x}[\mathcal{N}_i(\mathbf{w}),\ c_k B_K{:}(c_k{+}1)B_K)$
    $\mathbf{F} \leftarrow \mathbf{f}_i[c_k B_K{:}(c_k{+}1)B_K,\ b_n B_N{:}(b_n{+}1)B_N)$
    $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{F}$
  end for
  $\mathbf{Z}[b_m, b_n][r] \leftarrow \mathbf{Y}$ ▷ *partial to workspace*
  if $\textsc{LastArriver}(b_m, b_n)$ **then**
    $\mathbf{Y} \leftarrow \sum_{r'=0}^{S_k-1} \mathbf{Z}[b_m, b_n][r']$
    $\mathbf{y}[\mathbf{w},\ b_n B_N{:}(b_n{+}1)B_N) \leftarrow \mathbf{Y}$ **where** $M_{out}[\mathbf{w}]{=}1$
  end if
end for

5 Implementation

We implement WISEConv as a PyTorch extension with a Python frontend and a CUDA backend written in C++. A model-conversion pass replaces supported modules, preserves their parameters and weights, and statically marks worklist-reuse edges and rebuild points. The compute backend adapts CUTLASS [1] dense tiled implicit-GEMM pipelines for direct worklist consumption, covering both FP32 (SIMT) and FP16 (tensor core) arithmetic with NHWC tensors. The full-workload comparison uses the FP32 datapath; sensitivity studies additionally exercise the FP16 datapath. The backend supports standard, grouped, depthwise, and transposed convolutions, as well as the mask-aware auxiliary operators required by evaluated workloads. Each output carries a lightweight record that references its device-resident M_{out}, $\mathcal{W}$, and $n_{\mathcal{W}}$ buffers. All buffers are allocated during module initialization; mask selection remains with the upstream application.

The construction kernel maps one CUDA block to each construction tile in Algorithm 1. Warp ballots compute lane-local ranks, a short shared-memory prefix combines the warp counts, and one `atomicAdd` reserves the tile's contiguous worklist segment. Worklists use 32-bit linear coordinates and are provisioned for $|\mathcal{G}|$ entries; the runtime resets $n_{\mathcal{W}}$ on the operator's CUDA stream before each construction. When a learned gate supplies M_{out} directly, construction skips mask propagation and compacts active coordinates from the supplied mask. Compatible operators reuse the worklist and its length by aliasing the $\mathcal{W}$ and $n_{\mathcal{W}}$ buffers, while M_{out} always denotes the current operator's exact output mask.

The compute backend instantiates a C++ template family over (B_M, B_N, B_K); each template variant determines its persistent grid size from CUDA occupancy and the GPU's SM count. The partial-reduction workspace is shared across all layers at the maximum required size ($\max_l \max(S_k^l) \times |\mathbf{y}^l|$ accumulation-precision elements), allocated once during initialization. When a layer's policy table P^l selects the same mode for every tile count s, WISEConv eliminates the unused mode branches and partial-reduction logic from that layer's kernel, producing a streamlined variant that avoids the per-tile policy decode and workspace access. Each invocation launches its fixed persistent grid; the kernel reads device-resident $n_{\mathcal{W}}^l$, looks up P^l, and dispatches accordingly. Every backend variant agrees numerically with cuDNN at each active output position.

6 Evaluation

6.1 Experimental Setup

Platforms. We evaluate six GPU platforms. The four discrete GPUs are NVIDIA A100, RTX 4090, RTX 3080, and RTX 4070 Laptop. The embedded platforms are Jetson Xavier NX and AGX Orin; we also measure board energy on both. The discrete platforms use PyTorch 2.1.2, CUDA 12.1, and cuDNN 8.9.2; both Jetson platforms use PyTorch 2.1.0, CUDA 11.4, and cuDNN 8.6. All paths in the full workload comparison use FP32 (SIMT) with TF32 disabled. Targeted sensitivity studies also evaluate the device-resident Stream-K FP16 (tensor core) adapters on selected workload–platform pairs. On each platform, all backends run under the same fixed clock and power configuration.

Models, tasks, and mask sources. Our workloads span three vision tasks.

- **Event-based optical flow.** FireFlowNet [28] is a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy. We evaluate it on MVSEC [45], which contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [8], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks.
- **Detection.** We evaluate YOLOv8n and YOLOv8m [19] on MOT16 [25], a collection of crowded pedestrian videos for multi-object tracking. Their two model sizes expose how scale affects sparse-execution efficiency. For both models, DeltaCNN's temporal policy [30] thresholds activation differences between successive frames and propagates the resulting masks across layers.
- **Human pose estimation.** We use the DynConv pose model [37] on MPII [2], which depicts people performing diverse activities and annotates their 2D body joints. DynConv's learned spatial gates produce an input-dependent mask for each gated block from intermediate features, and MPII samples are temporally unrelated. We deliberately include this workload, where the device-resident Stream-K path consumes the exact worklist length without a temporal predictor, to isolate the worklist-driven execution model's latency gains from input-correlation assumptions.

Table 1 summarizes the workloads and evaluation scales. For every workload, calibration uses the training split, while all reported results use a disjoint test split. We use the authors' released weights for all four models [19, 28, 37]. Within each workload, we fix the inputs, weights, and upstream mask policy; every sparse path receives the same resulting layer masks, isolating the masked-convolution backend.

Table 1. Evaluation workloads.

	FireFlowNet [28]	**YOLOv8n/m** [19]	**DynConv** [37]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [8]	DeltaCNN [30]	DynConv [37]
Dataset	MVSEC [45]	MOT16 [25]	MPII [2]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

Baselines. We compare WISEConv against three FP32 (SIMT) execution paths established by prior work:

- **Dense** executes the complete model with the native PyTorch/cuDNN backend [6] with cuDNN autotuning enabled.
- **Tile skipping** uses DeltaCNN's default execution path for FireFlowNet with Ev-Conv and YOLO [30]. DynConv supplies output masks directly, so tile skipping bypasses only DeltaCNN's mask-propagation step.
- **Gather-scatter** uses DynConv's original CUDA backend for pose [37]. FireFlowNet and YOLO use MMCV's native MaskedConv2d CUDA path [5]; a thin wrapper only supplies the output mask and applies the workload-specific state update.

Measurements. We measure per-frame full-model GPU latency over the complete evaluation datasets. CUDA events bracket each frame's inference call; after workload-specific warm-up we run three complete rounds and average per-frame latency across all frames and rounds. The bracketed interval includes mask propagation, worklist construction, all network operators, and output update; it excludes input transfer, offline calibration and autotuning, and one-time state initialization. Legacy activity-mapped controls additionally perform nonblocking activity polling and bucket lookup in this interval; current Stream-K adapters read the device-resident count in the compute kernel and do not perform those host-side operations. We report η, T, and T_{eff} at the model level by accumulating required and issued convolution work across all layers within each frame before applying Eq. 3, then averaging across frames. On the Jetson platforms, we integrate board-level `VDD_IN` power over each frame's CUDA-event interval to obtain per-frame board energy, then average across frames. For accuracy, we follow the original workload protocols: AEE for optical flow [8], COCO-style average precision (AP) for detection, averaged over IoU thresholds from 0.50 to 0.95 [30], and PCKh for human pose [37]. Because MOT16 releases ground-truth detections only for its training split, AP on the test sequences uses fixed pseudo-labels from a dense YOLO26x teacher [20].

6.2 End-to-End Performance

Latency. WISEConv consistently converts upstream sparsity into lower full-model latency across all evaluated workloads and platforms. It is the fastest path in all 24 measured workload-platform pairs (Figure 3). Relative to the fastest evaluated competing path in each pair, it achieves a 1.57× geometric-mean speedup; individual-pair speedups range from 1.28× to 1.87×, corresponding to 22.1–46.5% lower latency. These gains persist despite a more than two-order-of-magnitude span in absolute latency, from 0.309 ms for FireFlowNet on RTX 4090 to 94.3 ms for YOLOv8m on Xavier NX.

Unlike existing sparse paths, WISEConv delivers uniform gains across workload regimes. Dense is the fastest evaluated competing path in 15 of the 24 pairs, compared with eight for tile skipping and only one for gather-scatter. FireFlowNet's low active ratio gives tile skipping enough headroom to

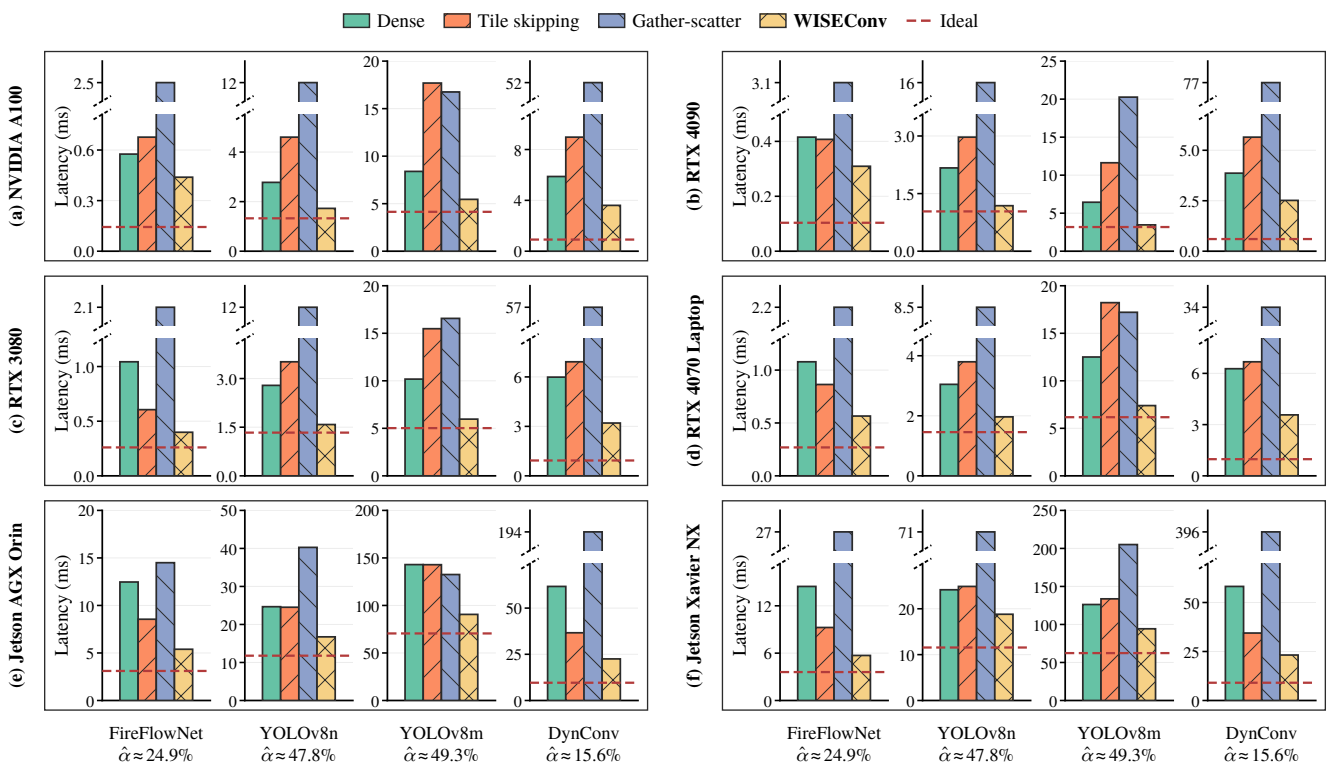


Figure 3. Full-model mean per-frame latency across GPU platforms. Each workload uses an independent y-axis. Labels report the model-level active ratio $\hat{\alpha}$, aggregated as described in §6.1 and averaged across evaluation sequences. Red dashed lines mark ideal proportional scaling, computed by applying each sequence's model-level active ratio to its dense latency before aggregation.

beat Dense on five of six platforms and reduce its latency by up to 42.0%. The higher active ratios of YOLOv8n and YOLOv8m leave less removable work: tile skipping reaches only near parity with Dense on AGX Orin and is slower in the other ten YOLO pairs. Gather-scatter becomes the fastest competing path only for YOLOv8m on AGX Orin, where the heavier convolution amortizes extraction and writeback. DynConv exposes the other limit: despite having the lowest active ratio, tile skipping beats Dense only on the two Jetson platforms. Sparsity magnitude alone therefore does not determine where an existing sparse path crosses the dense baseline.

WISEConv's lead over the per-pair fastest competing path is narrower on the embedded platforms (geometric mean 1.48× on Jetson vs. 1.62× on discrete GPUs), though it remains fastest in all eight Jetson pairs. The gap narrows because sparse paths themselves are more effective on Jetson: the fastest sparse path beats Dense in six of eight Jetson pairs but only three of 16 discrete-GPU pairs (geometric-mean speedup over Dense: 1.26× vs. 0.80×). Dense convolution is more latency-bound on the narrower Jetson GPUs, so the same active ratio yields a larger absolute latency reduction, making sparse paths more likely to outperform dense.

WISEConv also realizes more of the available sparsity headroom than every competing path. It is the closest path to the ideal proportional-scaling reference in every pair, running at 1.09–4.18× the reference while the fastest evaluated competing path remains at 1.88–6.43×. The remaining gap reflects work that does not shrink with convolution activity, including mask processing, worklist construction, and other non-convolution operators. DynConv lies at the upper end despite its low active ratio because every gated block still runs learned mask-prediction kernels and exposes exceptionally short worklists, as further discussed in §6.3.

Energy. WISEConv's latency gains also reduce per-frame board energy on both Jetson platforms. It is the lowest-energy path for every workload (Table 2), reducing energy by 17.0–39.8% relative to the lowest-energy competing path in each pair and by 28.5–72.6% relative to dense execution.

Accuracy. WISEConv's latency and energy gains do not alter the numerical output of the selected sparsity policies. For FireFlowNet and DynConv, its AEE and PCKh match the

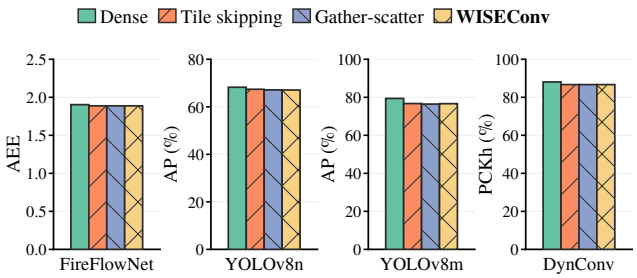


Figure 4. Task accuracy on RTX 3080 under the evaluated upstream sparsity policies, measured by AEE (↓), AP (↑), and PCKh (↑).

other sparse paths at the reported precision. In the YOLOv8 models, small floating-point differences accumulate across frames under temporal execution; even so, WISEConv's pseudo-label AP differs from the other sparse paths by at most 0.5% in relative terms (Figure 4), confirming output agreement across execution backends. Compared with dense execution, the upstream sparsity policies lower AP and PCKh by only 1.6–3.5%, while lowering FireFlowNet's AEE by 0.8%.

6.3 Microbenchmarks

Effective throughput. Under the same execution settings as the end-to-end evaluation, we estimate full-model throughput T and effective throughput ηT on RTX 3080 and AGX Orin (Figure 5). WISEConv sustains 30.7–87.2% of Dense raw throughput on RTX 3080 and 58.6–83.5% on AGX Orin, the highest raw throughput among the sparse paths in every workload. DeltaCNN's lower raw throughput reflects the impact of routing very sparse tiles through its fused selective path [30]. The two YOLO workloads lie at the upper end because their higher active ratios expose more parallelism and amortize fixed costs. Combining throughput and useful-compute ratio, WISEConv reaches 1.52–1.78× the strongest competing effective throughput on RTX 3080 and 1.45–1.65× on AGX Orin.

Table 2. Per-frame board energy (J) on the Jetson platforms. Parentheses report each sparse path's change relative to Dense.

GPU	System	FireFlowNet	YOLOv8n	YOLOv8m	DynConv
AGX Orin	Dense	0.26	0.48	2.70	1.20
	Tile skip	0.15 (-43.8%)	0.44 (-9.1%)	2.33 (-13.7%)	0.67 (-44.7%)
	Gather	0.29 (+9.5%)	0.70 (+46.1%)	2.42 (-10.4%)	2.37 (+97.0%)
	WISEConv	**0.10** (-61.6%)	**0.32** (-33.9%)	**1.57** (-41.7%)	**0.48** (-60.4%)
Xavier NX	Dense	0.28	0.49	2.58	1.08
	Tile skip	0.13 (-54.5%)	0.42 (-13.8%)	2.27 (-12.2%)	0.54 (-50.4%)
	Gather	0.33 (+17.0%)	0.85 (+75.8%)	3.18 (+23.2%)	2.83 (+160.9%)
	WISEConv	**0.08** (-72.6%)	**0.35** (-28.5%)	**1.78** (-31.1%)	**0.34** (-68.5%)

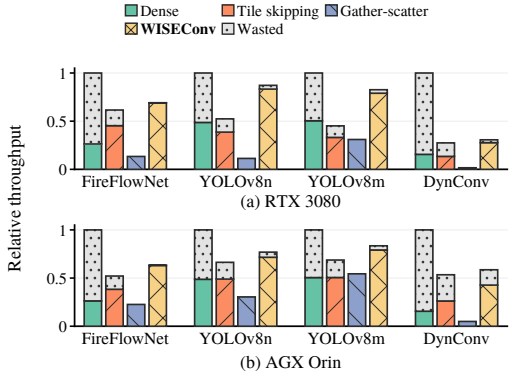


Figure 5. Full-model effective throughput ηT and wasted throughput $(1-\eta)T$ on RTX 3080 and AGX Orin, normalized within each platform to Dense raw throughput.

Table 3. WISEConv η decomposition and worklist-length statistics on RTX 3080. The n_W percentiles are computed from worklist lengths pooled across all layers and frames.

Workload	$\eta_{\text{construct}}$	η_{compute}	η	n_W		
				P10	P50	P90
FireFlowNet	99.9%	99.1%	98.9%	1,303	15,083	33,880
YOLOv8n	98.5%	96.8%	95.3%	77	937	5,937
YOLOv8m	98.7%	96.7%	95.5%	75	840	6,110
DynConv	100.0%	90.1%	90.1%	3	43	389

Table 3 localizes the remaining wasted work on RTX 3080. Its $\eta_{\text{construct}}$ stays at 98.5–100.0%, while the selected decompositions give η_{compute} = 96.7–99.1% for FireFlowNet and the two YOLO workloads. DynConv instead has η_{compute} = 90.1%: its low active ratio and small spatial grids yield exceptionally short worklists (P10, P50, and P90 lengths of 3, 43, and 389), making final-tile padding proportionally large. Because its inputs are temporally unrelated, DynConv uses the same exact-length device policy without a temporal activity selector. Even in this setting, its 90.1% useful-compute ratio is substantially above tile skipping's 48.9% on RTX 3080.

Latency cost decomposition. We decompose full-model YOLOv8n latency by execution stage on RTX 3080 and AGX Orin (Table 4). On the randomly sampled RTX 3080 frames, WISEConv reduces the convolution share from 2.55 to 1.10 ms while adding only 0.10 ms of construction; the corresponding sampled-row totals are 2.79 and 1.49 ms. The AGX Orin row shows the same balance at its slower operating point: construction accounts for 0.37 ms, while convolution falls from 22.61 to 13.41 ms and the sampled-row total falls from 25.09 to 17.07 ms.

Gather-scatter shows why exact selection alone is insufficient. Although it has the lowest convolution time on both

Table 4. Full-model YOLOv8n latency decomposition on RTX 3080 and AGX Orin. Entries are milliseconds, with shares of total latency in parentheses.

	System	Tot.	Cons.	Conv.	Elem.	Other
RTX 3080	Dense	2.79	—	2.55(91.4%)	0.14(5.0%)	0.10(3.6%)
	Tile skipping	3.38	—	3.05(90.3%)	0.16(4.6%)	0.17(5.1%)
	Gather	11.34	—	0.79(7.0%)	0.09(0.8%)	10.46(92.2%)
	WISEConv	1.49	0.10(7.0%)	1.10(74.0%)	0.12(8.1%)	0.16(10.8%)
AGX Orin	Dense	25.09	—	22.61(90.1%)	1.66(6.6%)	0.82(3.3%)
	Tile skipping	25.10	—	21.59(86.0%)	1.70(6.8%)	1.81(7.2%)
	Gather	39.87	—	9.14(22.9%)	0.94(2.4%)	29.79(74.7%)
	WISEConv	17.07	0.37(2.1%)	13.41(78.6%)	1.15(6.7%)	2.14(12.6%)

Notes. Cons., Conv., and Elem. denote worklist construction, convolution, and elementwise operators. Stage fractions from steady-state profiling are scaled to full-trace CUDA-event latency.

Table 5. YOLOv8n component ablation on RTX 3080; AGX Orin is reserved for the multi-platform refresh. Times are milliseconds.

Variant	RTX 3080				AGX Orin			
	η	Cons.	Conv.	Total	η	Cons.	Conv.	Total
Atomic append	95.6%	—	—	1.726	—	—	—	—
Global order	95.6%	—	—	1.759	—	—	—	—
No reuse	97.0%	—	—	1.726	—	—	—	—
DP only	95.5%	—	—	2.179	—	—	—	—
No Hybrid	95.5%	—	—	1.619	—	—	—	—
Sync exact selector	95.5%	—	—	6.207	—	—	—	—
Ours	95.5%	0.105	1.173	1.626	—	—	—	—

Notes. η is the full-model useful-compute ratio. Cons. and Conv. are construction- and convolution-stage latency; dashes indicate that variant-level stage attribution was not collected. AGX Orin entries are intentionally blank pending the multi-platform refresh.

platforms, *Other* accounts for 10.46 ms (92.2% of total latency) on RTX 3080 and 29.79 ms (74.7%) on AGX Orin. For gather-scatter, this category includes active-coordinate extraction, receptive-field materialization, GEMM setup, and scatter/writeback; for every path it also includes launch and dependency gaps. Direct worklist consumption avoids this auxiliary path while keeping convolution dominant.

6.4 Ablation Study and Sensitivity Analysis

Table 5 isolates the construction and consumption choices on YOLOv8n. On RTX 3080, Ours has 95.5% useful-compute ratio and a measured 1.626 ms latency. Atomic append, Global order, and No reuse raise latency by 6.1%, 8.1%, and 6.2%, respectively, exposing the cost of global coordination and rebuilding worklists. For the consumption stage, DP only is 34.0% slower than Ours, whereas disabling Hybrid is within 0.5% and is selected for several full-worklist shapes by the relaxed guard. Synchronizing the exact worklist length costs 6.207 ms (3.82× Ours), so its host readback dominates any gain from exact launch sizing. Cons. and Conv. are shown for Ours using the current semantic attribution; the remaining variant rows report their measured full-model latency and useful-compute ratio. AGX Orin entries are reserved for the multi-platform refresh.

Tensor core sensitivity. Table 6 compares the two arithmetic modes. For DynConv on RTX 3080, WISEConv reduces latency from 5.99 to 3.20 ms (1.87×) in FP32 (SIMT) and from 3.37 to 2.57 ms (1.31×) in FP16 (tensor core). On AGX Orin, it reduces FireFlowNet latency from 12.47 to 5.20 ms (2.40×) and from 4.31 to 2.78 ms (1.55×), respectively. Both settings therefore retain an end-to-end gain under tensor cores, although the gain narrows relative to FP32.

Construction does not cause the narrower gain: it remains only 0.08–0.09 ms (3.0–3.1% of WISEConv FP16 latency). Instead, tensor cores reduce the removable convolution time and expose workload-specific residual costs. In DynConv, the convolution saving falls from 3.07 ms in FP32 to 0.84 ms in FP16, while *Other* occupies 1.06 ms (41.1%) of the FP16 WISEConv path. In FireFlowNet, WISEConv still reduces FP16 convolution from 3.82 to 1.48 ms, after which elementwise operators account for 1.15 ms (41.5%) of its latency. FP16 PCKh for DynConv (88.07% Dense and 86.68% WISEConv) matches FP32 within 0.01 percentage points.

Stream-level batching. Table 7 shows how stream-level batching changes per-frame costs. At batch size 8, gather-scatter still spends 83.0% of its latency in *Other*, whereas tile skipping becomes the fastest competing path at batch sizes 4 and 8. Its launch grid is determined statically from the input shape, so increasing the batch dimension adds independent tiles from multiple temporal streams, consistent with DeltaCNN's observation for high-end GPUs [30]. The effect is concentrated in convolution: from batch size 1 to 8, tile skipping's per-frame convolution cost falls 2.68×, compared with 1.72× for WISEConv. WISEConv remains fastest, but its lead narrows from 1.76× to 1.55×. These results, in turn, show why adaptive decomposition matters at batch size 1: WISEConv adapts its work decomposition to each short worklist instead of relying on a larger batch to expand the launch grid.

6.5 Discussion

WISEConv consumes rather than generates masks, so its gains are conditional on the upstream policy and its activity distribution; it guarantees coverage of requested outputs but does not improve mask quality. Across our workloads, the evaluation shows that WISEConv most effectively converts the supplied mask sparsity into end-to-end latency reduction among the evaluated sparse paths. The adaptive decomposition is deployment-specific and must be recalibrated for a new model, device, or mask workload. Runtime dispatch reads the exact device-resident length. Only offline policy interpolation can affect performance; it cannot affect coverage or correctness.

The full workload comparison uses FP32 (SIMT): the evaluated masked-convolution baselines [5, 30, 37] provide only

Table 6. FP32 (SIMT) and FP16 (tensor core) full-model latency decomposition for DynConv on RTX 3080 and FireFlowNet on AGX Orin. Entries are milliseconds, with shares of total latency in parentheses.

Setting	Type	System	Tot.	Cons.	Conv.	Elem.	Other
DynConv RTX 3080	FP32	Dense	5.99	—	5.25(87.7%)	0.09(1.5%)	0.65(10.8%)
		WISEConv	3.20	0.09(2.9%)	2.18(68.2%)	0.07(2.1%)	0.86(26.7%)
	FP16	Dense	3.37	—	2.22(66.0%)	0.05(1.5%)	1.10(32.5%)
		WISEConv	2.57	0.08(3.0%)	1.38(53.4%)	0.06(2.5%)	1.06(41.1%)
FireFlowNet AGX Orin	FP32	Dense	12.47	—	11.82(94.9%)	0.62(5.0%)	0.02(0.2%)
		WISEConv	5.20	0.09(1.7%)	4.03(77.6%)	1.00(19.3%)	0.07(1.4%)
	FP16	Dense	4.31	—	3.82(88.7%)	0.48(11.0%)	0.01(0.2%)
		WISEConv	2.78	0.09(3.1%)	1.48(53.2%)	1.15(41.5%)	0.06(2.2%)

Notes. Stage fractions use the same semantic attribution as Table 4 and are scaled to each setting's full-trace CUDA-event latency.

Table 7. YOLOv8n latency decomposition under stream-level batching on RTX 3080. Each sequence is split into disjoint temporal streams; each batch combines one current frame per stream and advances their states independently. Stage definitions, units, and cell format follow Table 4.

	System	Tot.	Cons.	Conv.	Elem.	Other
Batch size=1	Dense	2.79	—	2.54(91.2%)	0.14(5.0%)	0.10(3.7%)
	Tile skipping	3.47	—	3.13(90.4%)	0.16(4.6%)	0.17(5.0%)
	Gather	11.33	—	0.75(6.6%)	0.09(0.8%)	10.50(92.7%)
	WISEConv	1.59	0.11(6.7%)	1.19(75.0%)	0.13(7.9%)	0.17(10.4%)
Batch size=4	Dense	1.83	—	1.61(88.0%)	0.15(8.0%)	0.07(4.0%)
	Tile skipping	1.66	—	1.41(84.7%)	0.12(7.4%)	0.13(7.8%)
	Gather	6.41	—	0.60(9.4%)	0.09(1.4%)	5.71(89.2%)
	WISEConv	1.03	0.03(3.1%)	0.78(76.5%)	0.09(8.8%)	0.12(11.6%)
Batch size=8	Dense	1.68	—	1.47(87.3%)	0.15(8.9%)	0.06(3.9%)
	Tile skipping	1.41	—	1.17(83.0%)	0.12(8.4%)	0.12(8.6%)
	Gather	3.80	—	0.56(14.7%)	0.09(2.3%)	3.15(83.0%)
	WISEConv	0.91	0.02(2.2%)	0.69(76.3%)	0.08(9.3%)	0.11(12.2%)

FP32 execution paths, so FP32 isolates execution-model differences from arithmetic-precision effects. The current FP16 adapter uses the same worklist contract with tensor cores. Split-K partitions accumulate FP32 partial sums and convert completed outputs to FP16. The sensitivity study measures where this implementation retains an end-to-end gain and where dense tensor cores absorb the available savings, without mixing FP16 sparse baselines into the FP32 comparison.

7 Conclusion

We presented WISEConv, a masked-convolution runtime that replaces the dense tiled pipeline's contiguous coordinate stream with an active-output worklist, computing only positions an upstream mask marks active while retaining the dense tensor layout and the regular per-position reduction. Construction completes in a single mask-space pass and is reused across compatible operators; consumption preserves tile-local ordering, bounds work with a device-resident worklist length, and adapts its decomposition to input activity without host synchronization. Our ablation isolates how these mechanisms control construction cost, worklist ordering, and synchronization overhead, while showing that their individual latency impact depends on device utilization. Together they secure both useful-compute ratio and throughput, converting upstream mask sparsity into end-to-end latency and energy reduction. WISEConv thus makes activity-driven scheduling a practical path to proportional latency reduction on deployed perception platforms.

References

[1] 2026. CUTLASS: CUDA Templates and Python DSLs for High-Performance Linear Algebra. https://github.com/NVIDIA/cutlass

[2] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.

[3] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.

[4] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.

[5] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).

[6] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).

[7] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[8] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. EvConv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.

[9] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739

[10] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense

and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117

[11] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712

[12] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

[13] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.

[14] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[15] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728

[16] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[17] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.

[18] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838

[19] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[20] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon, and Muhammet Esat Kalfaoglu. 2026. Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models. *CoRR* abs/2606.03748 (2026).

[21] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).

[22] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.

[23] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348

[24] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.

[25] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. *CoRR* abs/1603.00831 (2016).

[26] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR* abs/2104.08378 (2021).

[27] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. 2023. Stream-K: Work-centric Parallel Decomposition for Dense Matrix-Matrix Multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. doi:10.1145/3572848.3577479

[28] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.

[29] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.

[30] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.

[31] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426

[32] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.

[33] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. https://github.com/traveller59/spconv.

[34] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.

[35] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.

[36] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.

[37] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.

[38] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.

[39] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.

[40] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on

Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.

[41] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.

[42] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.

[43] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[44] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

[45] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR* abs/1801.10202 (2018).